

SAPIENCE · SECURITY REVIEW

Sapience LZ Composer Review.

A security review of LZ Composer Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Sapience	LZ Composer Review	Guardian	November 10 – November 11, 2025	November 18, 2025

§ 00

Contents.

16 pages — overview, confidence ranking, scope, 7 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 7 findings	P. 08
§ 05	Disclaimer	P. 15
§ 06	About Guardian	P. 16

§ 01

Overview.

Guardian reviewed LZ Composer Review across a single review round. The final report includes 5 Low, 2 Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	sapiencexyz/sapience
CLIENT	Sapience	METHODOLOGY	Smart contract · Manual · Main Review
PROJECT	LZ Composer Review	MAIN REVIEW COMMIT	de29bc5f4d7d
PREPARED BY	Guardian		
AUDIT WINDOW	November 10 – November 11, 2025		
FINAL REPORT	November 18, 2025		

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	0	0	0	0	0
Low	5	0	3	0	2
Informational	2	0	1	0	1
All severities	7	0	4	0	3

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed LZ Composer Review in `sapiencexyz/sapience`. The scope includes 356 source lines across 2 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
packages/protocol/src/predictionMarket/resolvers/PredictionMarketLZResolver.sol	158	215
packages/protocol/src/predictionMarket/resolvers/PredictionMarketLZResolverUmaSide.sol	198	279

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
L-01	Misleading getPredictionResolution Results	Validation	LOW	RESOLVED	p. 8
L-02	Lacking Receiver Settlement Check	Validation	LOW	RESOLVED	p. 9
L-03	Duplicate MarketIds Allowed	Validation	LOW	ACKNOWLEDGED	p. 10
L-04	Markets Allowed To Re-Resolve	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 11
L-05	Missing Configuration Updates	Configuration	LOW	ACKNOWLEDGED	p. 12
I-01	Incorrect MarketId Warning	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 13
I-02	Unnecessary marketAsserter	Superfluous Code	INFORMATIONAL	RESOLVED	p. 14

LOW **L-01**

RESOLVED

Misleading getPredictionResolution Results

CATEGORY	Validation
LOCATION	PredictionMarketLZResolver.sol
REVIEW	Main Review

— DESCRIPTION

When the `getPredictionResolution` function is called with an empty `encodedPredictedOutcomes` array, the function skips the loop and returns `(true, NO_ERROR, true)`. This contradicts `validatePredictionMarkets`, which reverts on zero markets. If any on-chain consumer relies solely on `getPredictionResolution`, an empty parlay incorrectly appears as a resolved success.

Furthermore, there is no maximum length validation as exists in the `validatePredictionMarkets` function.

— RECOMMENDATION

Consider mirroring the validation in `getPredictionResolution`: require the array to be non-empty and within configured bounds, or call `validatePredictionMarkets` internally and propagate its result.

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/827>

LOW **L-02**

RESOLVED

Lacking Receiver Settlement Check

CATEGORY	Validation
LOCATION	PredictionMarketLZResolver.sol
REVIEW	Main Review

DESCRIPTION

The prediction market side resolver does not enforce immutability of a market resolution once marked settled. A later cross-chain message with `assertedTruthfully=true` can set `resolvedToYes` to a different value, flipping an already-settled market outcome. While the configured UMA-side contract should only send one truthful resolution per market, any misconfiguration, replay from a different authorized remote bridge, or future code changes could toggle outcomes and corrupt payouts.

RECOMMENDATION

Consider adding validation to assert that `market.settled` is not already true in the `marketResolvedCallback` function.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/827>

LOW **L-03**

ACKNOWLEDGED

Duplicate MarketIds Allowed

CATEGORY	Validation
LOCATION	PredictionMarketLZResolver.sol
REVIEW	Main Review

— DESCRIPTION

Neither the validation nor resolution functions reject duplicate marketIds in the encoded PredictedOutcome[]. This can yield ambiguous parlay semantics, allow overcounting a single market multiple times.

— RECOMMENDATION

Track seen marketIds within the loop and reject duplicated entries. Alternatively, deduplicate before processing and enforce a strict normalized format.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Markets Allowed To Re-Resolve

CATEGORY	Unexpected Behavior
LOCATION	PredictionMarketLZResolverUmaSide.sol
REVIEW	Main Review

— DESCRIPTION

In the PredictionMarketLZResolverUmaSide resolver, the `assertionResolvedCallback` function always clears all mappings related to the `marketId`, which allows for subsequent submissions to the same market.

This is important to allow untruthful assertions to be re-submitted, however also allows assertions to be submitted for markets after they have already truthfully resolved.

— RECOMMENDATION

To avoid any unexpected behavior, consider only clearing the mappings when the assertion was resolved non-truthfully so that the already truthfully resolved market cannot be re-submitted.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Missing Configuration Updates

CATEGORY	Configuration
LOCATION	PredictionMarketUmaResolver.sol (005)
REVIEW	Main Review

DESCRIPTION

In the `PredictionMarketUmaResolver` contract there is no path to update the `config` values after it has been set on construction.

In case the configurations ought to be changed after deployment a new `PredictionMarketUmaResolver` instance will have to be deployed.

RECOMMENDATION

Be aware of this limitation for the `PredictionMarketUmaResolver` address and consider if a configuration setter should be added.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Incorrect MarketId Warning

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The UMA resolvers cannot validate the `marketId` correctness since they require resolution through the UMA oracle and the `marketId` is based upon the `keccak256(abi.encodePacked(claim, ":", endTime))` result.

As a result, matched predictions can pass the `_createPrediction` function even though they might use an incorrect `marketId`. Currently there is no pathway for users to be able to escape their matched prediction of an incorrect `marketId` was used, meaning that if no other market in the parlay resolves to false, and the invalid `marketId` remains unresolved the collateral for these users would be trapped.

RECOMMENDATION

Ensure that the `marketId` used to create orders from the offchain system is always correct and be aware of this risk.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-02**

RESOLVED

Unnecessary marketAsserter

CATEGORY Superfluous Code
LOCATION PredictionMarketLZResolverUmaSide.sol
REVIEW Main Review

DESCRIPTION

In the UMA side LZ Resolver the marketAsserter mapping is used to store the assertor of the marketId assertion, however this value is never used nor exposed to be read.

RECOMMENDATION

Consider removing the `marketAsserter` mapping.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/commit/d390e179e8d6a2ee1f9c634a64a188f5e199dfd3>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.